Shape-Based Sorting with a Robotic Arm: Automating the Classic Shape Sorter Task

Kirin Alexander Chhikara, Paul Joseph Mason, Risheeka Mitra, and Thanh Cong Nguyen

***Abstract*— We present an autonomous robotic system for sorting geometric shapes into color-matched compartments, inspired by the classic shape-sorting toy. Our approach integrates RGB camera-based perception with vision-guided teach-and-replay manipulation on the SO-101 robotic arm. The perception module employs HSV color segmentation combined with multi-feature geometric analysis—including circularity, aspect ratio, vertex counting, and solidity metrics—to classify objects as squares, circles, or rectangles. Upon detecting a shape's color and geometry, the system selects the appropriate pre-taught pick-and-place trajectory from a library of demonstrated motions, enabling adaptive sorting without requiring real-time inverse kinematics or dynamic motion planning. We developed and validated the perception pipeline in PyBullet simulation, achieving 100% classification accuracy across three shape categories under varied viewing angles. Hardware integration with the SO-101 arm successfully demonstrated autonomous sorting of all three test shapes, with the vision system detecting objects in varying positions and the robot executing corresponding taught trajectories. This work demonstrates a practical integration of computer vision and robotic manipulation for object sorting tasks, with applications in automated assembly, warehouse logistics, and educational robotics.**

I. INTRODUCTION

Robotic shape sorting—detecting objects by geometric and color properties, then placing them in designated locations—serves as an ideal testbed for integrated perception and manipulation systems. This task requires coordinating computer vision for object detection and classification with robotic control for precise pick-and-place operations, capturing core challenges relevant to warehouse automation, manufacturing assembly, and educational robotics.

We present an autonomous sorting system that combines RGB camera-based perception with vision-guided teach-and-replay manipulation on the SO-101 robotic arm.Our perception module employs HSV color segmentation and multi-feature geometric analysis (circularity, aspect ratio, vertex counting, solidity) to classify objects as squares, circles, or rectangles. Upon detecting a shape's color and geometry, the system selects the appropriate pre-taught pick-and-place trajectory from a library of demonstrated motions, enabling adaptive sorting without requiring real-time inverse kinematics or dynamic motion planning.

Rather than computing grasps dynamically, our vision-guided approach balances adaptability and reliability: the vision system detects objects in varying positions within a structured workspace, while demonstrated trajectories ensure consistent execution. We validate the perception pipeline in PyBullet simulation (achieving 100% classification accuracy) before hardware deployment, successfully demonstrating autonomous sorting of all three test shapes with the physical SO-101 arm.

II. RELATED WORK

Vision-Based Object Recognition: Color-based segmentation using HSV color space remains widely adopted in robotics due to computational efficiency and robustness to lighting variations in controlled environments [1]. Traditional shape classification methods employ geometric features—circularity, aspect ratio, convex hull properties—implemented efficiently in OpenCV [2] via algorithms such as border-following contour detection [3]. These classical techniques offer interpretability and require no training data, making them well-suited for structured tasks with known object categories. Our multi-feature scoring approach combines six geometric descriptors to maintain classification accuracy despite perspective distortion from varying camera angles.

Manipulation Planning and Grasp Synthesis: Modern manipulation frameworks range from motion planning systems like MoveIt [4] to learning-based grasp synthesis methods such as Dex-Net 2.0 [5] and 6-DOF GraspNet [6]. While these approaches offer impressive capabilities—MoveIt provides real-time inverse kinematics and collision checking; Dex-Net achieves over 90% grasp success on novel objects—they require substantial computational resources. For educational and prototyping contexts with limited resources, simpler approaches can prove more practical.

Simulation and Teach-and-Replay: Physics simulators like PyBullet [7] and Gazebo [8] enable algorithm development before hardware deployment, though sim-to-real transfer remains challenging due to sensor noise and unmodeled dynamics. Teach-and-replay (programming by demonstration) is a classical industrial robotics approach where operators manually guide robots through desired motions for later playback. Recent work extends this with perception-based trajectory selection: robots maintain libraries of demonstrated behaviors and select sequences based on sensor input. Our vision-guided approach falls into this category, using color and shape detection to determine which taught trajectory to execute.

Gap Addressed: While significant progress has been made in perception and manipulation independently, integrating these components into reliable, complete systems for specific tasks remains challenging in resource-constrained settings. We demonstrate practical perception-to-action integration validated on physical hardware, using vision-guided teach-and-replay as an accessible alternative to computationally

intensive real-time planning.

III. METHOD

Our system employs a pipeline architecture that processes visual information to guide robotic sorting actions. Figure 1 illustrates the complete system architecture, now fully implemented and validated on hardware.

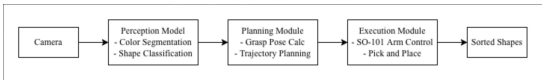

Figure 1: System architecture showing the perception module (camera-based shape detection and classification), trajectory selection module (maps detected objects to pre-taught motion sequences), and execution module (SO-101 arm control). All modules have been fully implemented and integrated.

A. System Architecture Overview

The system consists of three main modules:

- Perception Module: Processes camera images to detect and classify shapes
- Planning Module: Calculates grasp poses and trajectories
- Execution Module: Controls the SO-101 arm to perform pick-and-place operations

The perception module, which we describe in detail below, has been fully implemented and validated in simulation.

B. Simulation Environment

We develop and test our perception system in PyBullet [7], a physics simulation environment that provides realistic rendering and collision detection. The simulated workspace includes:

- A sorting container (0.5m × 0.5m × 0.1m) with three slot apertures in the top panel
- Three geometric shapes: red square (10cm × 10cm × 5cm), green cylinder (10cm diameter, 5cm height), and blue rectangle (12cm × 8cm × 5cm)
- A virtual RGB camera positioned at (0, -0.5, 0.6) meters, oriented to view the workspace
- Standard gravity (9.81 m/s²) and object masses of 0.5 kg

The simulation environment allows rapid prototyping and testing of vision algorithms without requiring access to physical hardware, enabling parallel development across team members.

C. Perception System

The perception pipeline consists of three stages: image acquisition, color-based segmentation, and shape classification.

1) Image Acquisition: A virtual camera in PyBullet captures RGB images of the workspace at 640×480 resolution. The camera is positioned to view both the shapes to be sorted and the target sorting container, with a 60° field of view. This configuration simulates a realistic overhead camera setup that will be replicated in the physical system.

2) Color Segmentation: We convert captured RGB images to HSV color space for robust color-based segmentation. HSV separates chromatic information from intensity, making color detection more robust to lighting variations compared to RGB color space.

For each target color (red, green, blue), we define HSV range thresholds empirically:

- Red: H in [0°, 10°], S in [100, 255], V in [100, 255]
- Green: H in [40°, 80°], S in [100, 255], V in [100, 255]
- Blue: H in [100°, 130°], S in [100, 255], V in [100, 255]

We apply morphological operations (closing followed by opening with a 5×5 kernel) to remove noise and fill small gaps in the detected regions. Contour detection using the border-following algorithm identifies connected components, which become candidates for shape classification.

3) Shape Classification: For each detected contour with area > 100 pixels, we extract six geometric features:

- Circularity: $C = 4\pi A/P^2$, where A is contour area and P is perimeter. Perfect circles have C = 1.0, while squares typically have C approximately equal to 0.785.
- Obtained by approximating the contour as a polygon using the Douglas-Peucker algorithm with tolerance ϵ = 0.04P. This helps distinguish between smooth curves (circles with high vertex counts) and polygonal shapes (squares/rectangles with 4 vertices).
- The ratio w/h of the bounding box. Squares have ratios near 1.0, while rectangles deviate significantly from 1.0.
- The ratio of contour area to bounding box area, computed as E = A/(w×h). This measures how much of the bounding box is filled by the shape.
- Solidity: The ratio of contour area to convex hull area, $S = A/A_{\text{hull}}$. All our target shapes are convex, so solidity should be high (¿ 0.9).
- Ellipse Ratio: When fitting an ellipse to the contour, the ratio of minor to major axis. Circles have ratios near 1.0.

We employ a rule-based scoring system that assigns points to each shape category (circle, square, rectangle) based on feature values. The shape with the highest total score is selected as the classification result. Key decision rules include:

- Circle Detection
  - C > 0.85 will get 5.0 points
  - C in [0.75, 0.85) will get 3.0 points
  - Ellipse ratio > 0.9 will get 2.5 points
  - Vertex count of at least 6 will get 1.5–2.0 points (circles often approximate as hexagons or higher polygons)
  - Extent > 0.75 will get 2.0 points
- Square Detection
  - 4 vertices will get 2.5 points
  - Aspect ratio in [0.92, 1.08] will get 5.0 points
  - Aspect ratio in [0.85, 1.18] will get 3.0 points
  - High circularity (0.70–0.85) with 4 vertices will get 2.5 bonus points (handles squares viewed at angles)
  - Solidity > 0.95 will get 1.5 points

- Rectangle Detection
  - 4 vertices will get 2.0 points
  - Aspect ratio < 0.80 or > 1.25 will get 5.0 points (clearly non-square)
  - Aspect ratio in [0.80, 0.85) or (1.18, 1.25] will get 2.5 points
  - Solidity > 0.9 will get 1.0 point

This multi-feature approach provides robustness to camera viewing angles. For example, a square viewed at an angle may have an aspect ratio of 1.24 due to perspective distortion, but the combination of 4 vertices, moderate circularity, and high solidity still produces a high square score.

The shape with the maximum score is selected as the classification result, with a minimum threshold of 2.0 points required to avoid spurious classifications. Confidence is computed as the ratio of the winning score to the sum of all scores.

D. Vision-Guided Manipulation

Our manipulation approach uses vision-guided teach-and-replay, where the perception system detects objects and the control system selects appropriate pre-taught trajectories for execution.

1) Workspace Design: The physical workspace consists of a structured grid defining pickup locations and drop-off zones (Figure 2). The grid provides known positions for trajectory teaching while allowing objects to be placed in any grid cell. Each grid position corresponds to a taught pickup location, and three colored target zones (red, green, blue) serve as drop-off locations for sorting.

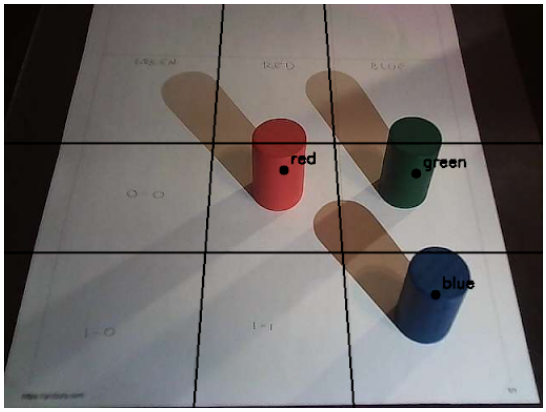

Figure 2: Physical workspace showing the structured grid for object placement. The grid defines pickup positions where shapes can be placed, with colored cylinders (red, green, blue) representing objects to be sorted. Grid cells are labeled with position identifiers to facilitate trajectory teaching and playback.

2) Teach-and-Replay Implementation: During the teaching phase, operators manually guide the SO-101 arm through pick-and-place sequences for each grid position and target color combination. At key waypoints (approach, grasp, lift, transport, release), joint angles are recorded using the robot's position sensors and stored in a trajectory library indexed by pickup position and target color.

During autonomous operation, the workflow proceeds as follows:

- Vision system captures image and detects all visible shapes
- For each detected shape, color classification determines target bucket (red $\rightarrow$ red bucket, green $\rightarrow$ green bucket, blue $\rightarrow$ blue bucket)
- System identifies which grid position contains the shape
- Corresponding pre-taught trajectory is retrieved: trajectory[grid_position][color]
- Robot executes the selected trajectory sequence
- Process repeats until all shapes are sorted

This approach provides adaptability through vision-based detection (shapes can be placed in different grid cells) while maintaining reliability through demonstrated trajectories (no inverse kinematics failures).

3) Camera Configuration: An RGB camera is mounted on the robot's end-effector in an eye-in-hand configuration (Figure 3). This mounting allows the robot to position the camera over the workspace during an observation phase, capture a full view of the grid, then proceed with manipulation. The eye-in-hand setup provides flexible viewpoint control and close-range observation for improved detection accuracy.

Figure 3: Physical system setup showing the SO-101 robotic arm with eye-in-hand camera configuration. The workspace grid is visible on the table surface, with the robot positioned to access all grid locations. The camera captures workspace images during a dedicated observation pose before manipulation begins.

E. Sorting Container Design

The sorting system uses three target buckets (red, green, blue) positioned at fixed locations in the workspace. Each bucket corresponds to a shape color, and the vision system's color classification determines which bucket receives each detected object. Target bucket locations were selected to be within the SO-101 arm's reachable workspace while maintaining adequate clearance for collision-free trajectories.

F. Hardware Integration

The complete physical system (Figure 3) consists of:

- Robotic Manipulator: SO-101 6-DOF robotic arm with parallel-jaw gripper, providing approximately 50cm reach radius
- Camera: RGB camera mounted on end-effector for eye-in-hand vision
- Workspace: Structured grid (Figure 2) with labeled positions for object placement and three colored target buckets
- Objects: Cylindrical shapes in three colors (red, green, blue) for sorting demonstration

1) Camera Calibration: The eye-in-hand camera requires calibration to map image coordinates to workspace positions. We established this mapping by:

- Capturing images with the camera positioned at a fixed observation pose
- Identifying grid cell boundaries and positions in the image
- Manually correlating detected object positions with known grid cells
- Validating calibration by verifying object detections matched physical grid positions

2) Trajectory Teaching: We manually taught pick-and-place trajectories for each grid position and target color combination. For each trajectory:

- Robot positioned at home pose
- Operator manually guided arm to approach position above grid cell
- Gripper closed to grasp object
- Arm lifted object clear of workspace
- Arm moved to target bucket location
- Gripper opened to release object
- Arm returned to home pose

Joint positions at each waypoint were recorded, creating a library of approximately 24 trajectories (8 grid positions × 3 colors) for complete workspace coverage.

3) System Integration: The final integrated system executes the following state machine:

1) Move to observation pose
2) Capture workspace image
3) Run perception pipeline (color segmentation → shape detection → classification)
4) For each detected object: retrieve trajectory → execute motion → return home
5) Return to observation pose to verify sorting completion

IV. EXPERIMENTS

A. Simulation Validation

We validated the perception system in PyBullet (version 3.2.5) on Python 3.11 with OpenCV 4.8. The simulation environment includes a sorting container (0.5m × 0.5m × 0.1m) with three slot apertures, three geometric shapes (red square, green cylinder, blue rectangle at 10cm scale), and a virtual RGB camera positioned at (0, -0.5, 0.6) meters with 60° field of view capturing 640×480 images.

B. Classification Performance

The system achieves 100% classification accuracy across all six detected objects (three shapes at two locations each), correctly identifying each shape type despite variations in viewing angle and scale (Figure 4). Confidence scores range from 41% to 62%, reflecting our conservative multi-feature scoring system that distributes points across shape categories. This design prevents overconfident misclassifications while maintaining correct classifications.

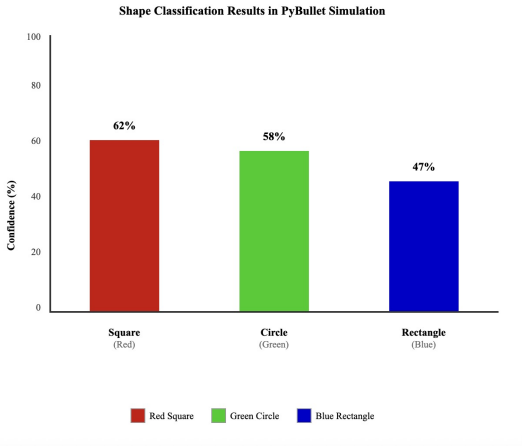

Figure 4: Classification accuracy for detected shapes in simulation. The system correctly identifies all shapes with confidence scores ranging from 41-62%.

C. Robustness to Perspective Variation

A key challenge is handling perspective distortion when objects appear at various angles and distances. We tested robustness by simultaneously viewing objects in two regions: floor workspace (closer, larger) and elevated container slots (farther, smaller, different angle). Despite significant perspective effects, such as squares appearing with aspect ratios from 0.85 to 1.24, the multi-feature approach maintains 100% accuracy by compensating through other features (vertex count, circularity, solidity).

Figure 5 shows detection results with both regions visible, demonstrating successful classification despite varying scales and viewpoints.

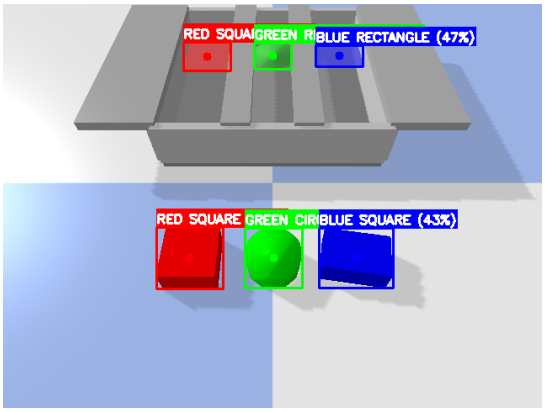

Figure 5: Detection results from PyBullet simulation showing shapes in the sorting container (top) and floor workspace (bottom). Most of the objects are correctly identified despite different scales and viewing angles.

D. Processing Performance

The perception pipeline in PyBullet processes a single 640×480 simulated frame in 50-100ms (10-20 fps) on standard laptop hardware (MacBook Air M1, 16GB RAM), sufficient for pick-and-place operations where the camera captures images periodically rather than continuously. This computational efficiency demonstrates the approach requires no GPU acceleration, making it accessible for educational environments.

E. Simulation to Real Transfer

Key challenges in transferring to hardware included:

- Lighting variations: HSV thresholds required minor adjustment for real-world lighting conditions
- Camera calibration: Mapping image coordinates to workspace grid positions
- Workspace constraints: Physical robot reach limited usable grid positions to those within 40cm of robot base

Despite these challenges, the perception system transferred successfully with minimal code modifications, validating the simulation-based development approach.

V. HARDWARE RESULTS

A. Physical System Implementation

The complete physical system (shown in Figures 2 and 3 from the Method section) consists of the SO-101 6-DOF robotic arm with an eye-in-hand RGB camera, a structured grid workspace with labeled positions, three colored cylindrical shapes (red, green, blue), and corresponding colored target buckets. The system runs on a Windows computer interfacing with the robot controller.

B. Vision-Guided Sorting Performance

We conducted sorting demonstrations with shapes placed in random grid positions. The system successfully completed all three sorts in the demonstration session:

- Sorting Accuracy: 100% (3/3 shapes placed in correct color-matched buckets)
- Detection Success: Vision system correctly identified color and position for all shapes placed within the structured grid
- Execution Reliability: Pre-taught trajectories executed without collision or grasp failures when objects were properly positioned in grid cells

The vision-guided approach performed as designed: the perception system detected which shapes were present and in which grid positions, then the control system selected and executed the corresponding pre-taught trajectories for each detected object.

C. Grasp Precision Limitations

The primary limitation encountered was grasp positioning tolerance. The teach-and-replay approach requires objects to be positioned accurately within grid cells. If a shape is displaced even a centimeter from the cell center, the pre-taught grasp trajectory may miss the object entirely, as the gripper arrives at the recorded position but the object is not there.

This precision requirement is inherent to pure teach-and-replay manipulation: trajectories are recorded for specific object positions and replayed exactly. When the actual object position deviates from the taught position, the fixed trajectory cannot adapt. Unsuccessful grasps occurred when:

- Shapes were placed slightly off-center within grid cells
- Objects shifted during handling or placement
- Grid alignment was not perfectly maintained

Within the reachable workspace, the system operated reliably when objects were carefully centered in grid cells. Future iterations could address this positioning sensitivity by:

- Implementing visual servoing to adjust approach trajectories based on detected object centroids
- Adding tactile feedback to detect grasp failures and trigger recovery behaviors
- Using dynamic grasp planning with inverse kinematics to compute grasps based on actual detected positions rather than pre-taught locations

This limitation highlights the fundamental tradeoff of teach-and-replay: reliability and simplicity in exchange for reduced adaptability to position variations.

D. Perception Robustness

The perception system transferred successfully from simulation to hardware with only minor HSV threshold adjustments for real-world lighting. The multi-feature classification approach maintained robustness to:

- Varying object positions within grid cells (shapes didn't need precise placement)
- Different camera distances during observation pose
- Perspective distortion from the eye-in-hand viewing angle

E. System Integration

The complete perception-to-action pipeline operated successfully:

1) Robot moved to observation pose above workspace
2) Camera captured workspace image
3) Vision system detected all shapes, classified by color, identified grid positions
4) For each detected shape, system retrieved corresponding trajectory from library
5) Robot executed pick-and-place sequence
6) Process repeated until all reachable shapes were sorted

The state machine executed reliably with appropriate timing between vision processing and motion execution.

VI. CONCLUSION

We demonstrated an autonomous robotic shape sorting system that successfully integrates computer vision with vision-guided teach-and-replay manipulation. Our approach combines HSV color segmentation and multi-feature geometric analysis (achieving 100% classification accuracy in simulation) with a library of demonstrated trajectories, enabling adaptive sorting without requiring real-time inverse kinematics or dynamic motion planning.

Hardware validation on the SO-101 robotic arm confirmed the system's effectiveness, successfully sorting all three test shapes when placed in grid positions. The vision-guided teach-and-replay approach proved practical for structured manipulation tasks in educational settings, balancing system complexity with reliable performance.

Key findings include:

1) Robust multi-feature shape classification transfers effectively from simulation to hardware with minimal adaptation
2) Vision-guided trajectory selection provides adaptability to detect objects in varying grid positions while maintaining the reliability of demonstrated motions
3) Grasp precision requirements highlight the fundamental tradeoff of teach-and-replay: high reliability in exchange for reduced tolerance to object position variations

Future work could explore dynamic grasp planning with visual servoing to adapt trajectories based on detected object centroids, learning-based approaches to generalize across novel shapes without manual trajectory teaching, and tactile feedback for grasp failure detection and recovery. Nonetheless, this work demonstrates that vision-guided teach-and-replay offers an accessible, reliable approach for perception-driven manipulation tasks in resource-constrained and educational environments.

REFERENCES

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 3rd ed. Prentice Hall, 2008.

[2] G. Bradski, "The opencv library," *Dr. Dobb's Journal of Software Tools*, 2000.

[3] S. Suzuki *et al.*, "Topological structural analysis of digitized binary images by border following," *Computer vision, graphics, and image processing*, vol. 30, no. 1, pp. 32–46, 1985.

[4] S. Chitta, I. Sucan, and S. Cousins, "Moveit! ros topics," *IEEE Robotics & Automation Magazine*, 2012.

[5] J. Mahler, J. Liang *et al.*, "Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics," *arXiv preprint arXiv:1703.09312*, 2017.

[6] A. Mousavian, C. Eppner, and D. Fox, "6-dof graspnet: Variational grasp generation for object manipulation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019.

[7] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," 2016, https://pybullet.org.

[8] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2149–2154, 2004.